

eGov PH Hackathon 2026

eHakbang

"Your step-by-step government journey, now automated."

Technical Q&A prep sheet — anticipated judge/panel questions and grounded answers, based on the current codebase.

Product	eHakbang — AI government journey planner & automation assistant
Stack	Next.js 16 (App Router) · React 19 · TypeScript · Tailwind CSS 4
AI	OpenAI Responses API (web search + structured outputs)
Integrations	eGov Face Liveness · eGovPay · eMessage (SMS) · Supabase
Prepared	2026-07-27

Contents

1. Product & Problem
2. Architecture & Tech Stack
3. AI / Journey Generation
4. Government API Integrations
5. Payments (eGovPay)
6. Data Model, Persistence & Privacy
7. Mocking / Simulation Strategy
8. Security
9. Testing & Quality
10. Hard / Curveball Questions

1. Product & Problem

Q: What problem does eHakbang solve?

A: After a life event (getting married, having a baby, losing a job, retiring, starting a business, a death in the family, etc.), Filipinos face scattered obligations and entitlements across many agencies (SSS, PhilHealth, Pag-IBIG, BIR, PSA, DOLE, GSIS, DSWD, LGU/barangay, COMELEC). Information is fragmented, the correct order is unknown, benefits go unclaimed, and even an informed citizen still has to queue, pay, and file in person at multiple offices. The existing eGov PH chatbot names agencies but doesn't sequence steps, list documents, track completion, or act on the citizen's behalf.

Q: What does eHakbang actually do, end to end?

A: Given a life event (typed free-text or picked from a card), it generates an ordered, plain-language checklist of government steps — each with the responsible agency, why it's needed, documents required, estimated time, any fee, and an official link. The citizen can complete steps manually (default) or opt into per-step automation: paying a fee in-app via eGovPay, queuing an auto-apply submission, and getting notified. It never requires an account or sign-in.

Q: Who is the target user?

A: Every Filipino who just went through a qualifying life event, with priority on non-tech-savvy users — e.g. a newly married person, a new/informal-worker mother, a retiring employee, or a barangay officer assisting constituents on their behalf.

Q: What's the core value proposition in one line?

A: "eHakbang tells Filipinos the right government steps in the right order, then does the paperwork for them" — turning a fragmented, in-person, multi-agency process into a single guided (and optionally automated) mobile-first flow.

Q: Why is this a checklist app and not just another chatbot?

A: A chatbot answers questions one at a time and doesn't retain structure. eHakbang produces a structured, ordered, resumable journey object (steps, fees, documents, progress) that persists across sessions and can be acted on — checked off, paid for, or automated — not just read.

2. Architecture & Tech Stack

Q: What's the stack?

A: Next.js 16 (App Router, React Server Components + Route Handlers) with React 19 and TypeScript, styled with Tailwind CSS 4. Supabase (Postgres, service-role client) is used server-side as a cache/store. Vitest + Testing Library for unit/component tests. No separate backend service — all server logic lives in Next.js Route Handlers under `app/api/*` and server-only modules under `lib/server/*`.

Q: How is client vs. server code separated?

A: Every module that touches a secret (API keys, Supabase service role) is tagged with the `"server-only"` import at the top (e.g. `lib/server/liveness.ts`, `lib/server/egovpay.ts`, `lib/supabase/server.ts`) — this makes it a build error if such a module is ever imported from client (`"use client"`) code, so credentials structurally cannot reach the browser. Route Handlers in `app/api/*` are the only callers.

Q: Where does journey state live — is there a database of users?

A: No user database and no accounts at all. A journey lives in the browser's `localStorage` (see `lib/journey-store.ts`), created only once the user actually engages (e.g. marks a step done), so a fresh install shows a clean slate. Supabase is used only as a server-side cache for AI-generated journey content per life event (not per user) and for the mocked auto-apply queue — neither table is keyed by a person.

Q: Why localStorage instead of a real account system?

A: It's a deliberate product principle (PRD: "No sign-in", "Data minimization") — the app should work with zero friction and zero registration. It also sidesteps a large class of PII-handling and auth-security concerns for a hackathon-scoped build.

Q: What happens if the user clears their browser storage or switches devices?

A: The journey is lost — this is a known, accepted trade-off of the no-account model. There's a small migration helper (`migrate()` in `journey-store.ts`) that backfills newly-added fields onto already-stored journeys so schema changes don't crash old `localStorage` entries, but there's no cross-device sync.

Q: Is the UI localized?

A: Yes — a Filipino/English toggle (`lib/i18n.tsx`) drives both the static UI copy and the language passed to the AI generation/Q&A calls, so generated journey text and chat answers come back in the selected language, not just the UI chrome.

3. AI / Journey Generation

Q: How are the checklist steps generated — hardcoded or AI?

A: AI-generated, via OpenAI's Responses API with the `web_search` tool (`lib/server/openai-journey.ts`), constrained to a strict JSON schema mirroring the app's `JourneyStep` type. The model is instructed to search official `.gov.ph` sources and never invent a fee, deadline, or document it didn't find. A hand-written seed list (`lib/event-journeys.ts`) exists purely as an offline fallback.

Q: Why web search instead of just prompting the model's own knowledge?

A: Government fees, document lists, and procedures change and the model's training data goes stale. Grounding each step in a live web search over official agency sites keeps amounts and requirements current and gives a citable `official_source_url` instead of a hallucinated number.

Q: How do you stop the model from making up a fee or a required document?

A: Three layers: (1) explicit system-prompt rules forbidding invented fees/documents; (2) a strict JSON schema where `fee` is either `null` or a fully-specified object with a mandatory (possibly-null) `official_source_url`; (3) downstream, `lib/journey-fees.ts` parses the free-text fee amount conservatively — anything that isn't a clean number or a recognized "free" phrase (a range, a qualifier, prose) is treated as unparseable and routed to "pay manually at the agency" rather than guessed, since an under/over-charge is worse than asking the citizen to pay in person.

Q: Isn't this expensive/slow to call an LLM+web-search on every visit?

A: Results are cached in Supabase per `(event_id, language)` — not per user — with a 24-hour staleness check done lazily on read (`lib/server/journey-requirements.ts`): a cache hit skips the model call entirely, and only a request past a 24h-old cached row (or a cache miss) triggers regeneration.

Q: What happens if OpenAI or Supabase is down / misconfigured?

A: The lookup is wrapped in a try/catch that falls back to the hand-written seed journey in `lib/event-journeys.ts` for that life event — so the app keeps working, in a degraded (non-freshly-generated) form, with zero API keys configured at all.

Q: How does the "documents_required" vs "required_fields" split work?

A: `documents_required` is only for things the citizen uploads (certificates, IDs, forms). `required_fields` is for data values only the citizen knows that no document supplies — e.g. a proposed business name or a bank account number — each with a `field_key`, label, type (text/number/date), hint, and required flag. The prompt explicitly tells the model not to duplicate an item across both lists.

Q: Is there a chat feature, and how is it different from journey generation?

A: Yes — a per-step "Ask about this step" Q&A (`lib/server/ask-step.ts`, backing `components/journey/AskAboutStep.tsx`). It deliberately uses a lighter/faster model (`gpt-4.1-mini` by default) with no web search and no structured output — just a short plain-text answer scoped to that one step's already-generated context (title, agency, documents, fee, etc.), instructed to say "I don't know, check the official link" rather than invent an answer outside that context.

Q: How do you keep steps in the right order across very different life events?

A: The generation prompt explicitly asks the model to "order steps in the sequence a citizen should realistically complete them" as part of the same structured call — ordering isn't a separate pass, it's baked into how the JSON array is produced.

Q: Does eHakbang tell users their exact benefit amount?

A: No — this is a deliberate "honest AI" principle. Benefit amounts/eligibility are never presented as guaranteed; every fee/amount is cited back to an official source URL, and step-level Q&A is instructed to defer to the official link rather than assert a number it isn't certain of.

4. Government API Integrations

Q: Which real eGov PH APIs does the app integrate with?

A: Three hackathon-provided APIs, each with its own server-only client module: the eGov Face Liveness API (`lib/server/liveness.ts`), eGovPay payment gateway (`lib/server/egovpay.ts`), and eMessage SMS (`lib/server/emessage.ts`). Journey generation itself runs on OpenAI rather than a separate "eGov AI" endpoint — the PRD names it eGov AI, but under the hood it's OpenAI's Responses API playing that role for the demo.

Q: What is the Face Liveness check used for?

A: It gates the payment flow: before a fee-bearing step's payment can be started, the citizen completes a face-liveness capture; the resulting session token is verified server-side (status `SUCCEEDED` and `confidence_score >= 95.0`) before `/api/payment` will create an eGovPay transaction. It's a lightweight "prove you're a live person, not just clicking a link" gate rather than a full identity/KYC check.

Q: Can the liveness check be bypassed by calling the payment API directly?

A: No — `/api/payment` re-verifies the liveness token server-side itself (calling `getLivenessResult`) rather than trusting a client-side "verified" flag, specifically because a client-only gate is bypassable by hitting the route directly.

Q: How does eMessage (SMS) get used?

A: To send status notifications/confirmations to a phone number the citizen optionally supplies at the point of notification (no account, no phone number stored otherwise) — e.g. auto-apply status updates. It's a thin POST to `{base_url}/messaging/v1/sms/push` with an auth header, matching the same server-only-module pattern as the other integrations.

Q: Is there a tamper-evident receipt / blockchain anchoring feature (eGovChain), as the PRD mentions?

A: It's specified in the PRD (FR-18) as part of the automation results dashboard, but it is not present in the current codebase — there's no `eGovChain` client or anchoring call implemented. If asked live, be upfront that it's PRD-scoped/future work, not shipped.

5. Payments (eGovPay)

Q: How does the in-app fee payment flow work?

A: `/api/payment` takes an `eventId` , the step numbers to pay, and a liveness token. It re-derives the canonical journey server-side (the same source `/api/journey` uses), recomputes the fee bill itself via `getFeeBill` , and only accepts step numbers that independently come back as payable — the client-submitted amount is never trusted. It then creates an eGovPay transaction and returns a redirect URL; `/journey/pay/callback` confirms success by polling `GET /api/payment/[uuid]` , keyed off the unambiguous `paid_at` field.

Q: Why re-derive the fee bill server-side instead of trusting the amount the client sends?

A: Because fee amounts are AI-generated free text tied to a specific journey/event, not client state — trusting a client-supplied amount would let anyone request an arbitrary transaction amount. Recomputing from the same cached/generated journey the checklist itself uses closes that gap.

Q: How is the eGovPay request signed?

A: An HMAC-SHA256 digest over `"{amount}|{txnid}"`, keyed by the eGovPay API token. One integration detail worth knowing: verified against the real sandbox, the HMAC key is the token with its `test_` prefix stripped (even though the full `test_...` string is still sent as the `X-eGovPay-Token` header), and the amount must be the minimal numeric string form (e.g. `"155"` or `"155.5"`) rather than forced to two decimals like `"155.00"` — getting either of those wrong produces a digest mismatch.

Q: Not every fee is a fixed number — how do you handle ranges like "₹200–500 depending on LGU"?

A: `parseFeeAmount` only accepts a plain currency-prefixed number or a recognized zero-fee phrase ("free", "n/a", "no fee", etc.); anything else (a range, a qualifier) returns `null` and that step is bucketed into `manualPayNotes` — shown to the citizen as "pay this one in person" instead of being silently mis-billed.

Q: Do you verify eGovPay's webhook is genuinely from eGovPay?

A: Not yet — `/api/payment/webhook` currently just logs and accepts the payload; there's an explicit TODO noting eGovPay's docs don't specify a signature scheme for the webhook, and that verification should be added once documented. It isn't the UI's source of truth for payment success anyway — the callback page confirms via polling `GET /api/payment/[uuid]`, which hits eGovPay directly.

6. Data Model, Persistence & Privacy

Q: What does the core data model look like?

A: A `Journey` (life event, summary, ordered `JourneyStep[]`, status, language, timestamps, and per-user progress: completed/paid/auto-applied/claimed step numbers, plus citizen-supplied `field_answers`). Each `JourneyStep` carries agency, step type (`record_update` vs `benefit_claim`), reason, documents, estimated time, an optional `fee`, an eGov catalog lookup term, optional `required_fields`, and an optional `fulfills_id` linking it to an ID-wallet entry.

Q: What is the "ID Wallet" feature?

A: A local, on-device record of which government ID *types* the citizen already holds (TIN, SSS, PhilHealth, PhilSys, etc.) — never ID numbers, names, or documents, just presence flags in `localStorage`. If a step's purpose is to obtain an ID already recorded in the wallet, that step is auto-satisfied without manual action, keeping the "zero personal data" posture intact.

Q: What data does eHakbang actually collect and store server-side?

A: Per the PRD's data-minimization principle: nothing tied to an individual beyond what an in-flight transaction requires. Supabase stores AI-generated journey content keyed by life-event id + language (not by user) and the mocked application-queue rows (step + field answers a citizen typed for that step, not stored elsewhere). No accounts, no persistent user profile table, no document/image storage.

Q: Two step-verification screens exist (document upload, liveness) — what do they actually check right now?

A: `lib/verification.ts` is explicitly labeled a flow prototype: `VERIFICATION_REQUIRED_EVENT_IDS` is currently empty, so no life event is gated by it today, and even where wired up, the document-upload step auto-accepts with no real validation. It exists to demo the UX shape; the file's own comment flags that wiring real document/biometric checks would introduce PII handling the current zero-data design doesn't yet account for, so that should be revisited before any real rollout.

7. Mocking / Simulation Strategy

Q: What in this app is real vs. simulated?

A: Real: OpenAI journey generation and step Q&A, the eGov Face Liveness API (sandbox), eGovPay (sandbox), eMessage SMS, and Supabase caching. Simulated: (1) the "auto-apply" agency submission — there's no real agency-facing backend, so a queued application is just flipped from `pending` to `accepted` after a fixed 15-second delay, checked lazily on read; (2) the document-upload verification step (auto-accepted, no real validation); (3) an optional dev-only Face Liveness mock for when the real hackathon API is unreachable during development.

Q: Why fake the agency-submission backend instead of leaving it unimplemented?

A: There is no real write-endpoint into SSS/PhilHealth/Pag-IBIG/etc. for a hackathon to call — no such API exists for external integrators. Simulating acceptance after a short delay lets the full UX (queued → status polling → accepted → notify) be demoed end-to-end, matching the PRD's explicit requirement (FR-22) that simulated integrations be clearly labeled so no one mistakes it for a real government submission.

Q: How does the Face Liveness dev-mock work, and is it reachable in production?

A: `isLivenessMockEnabled()` in `lib/server/liveness.ts` is hardcoded to return `false` whenever `NODE_ENV === "production"`, full stop — so the mock path is structurally unreachable in a real deployment regardless of any other setting. In development it auto-activates if the real API's base URL/key aren't configured, or can be forced on via `LIVENESS MOCK=true`. When active, session-creation redirects to an in-app `/dev/liveness-mock` page instead of the real eGov capture page, and result-fetching auto-returns a `SUCCEEDED` /99.9% result — the rest of the flow (start → connecting → callback) is unchanged so the UI is exercised identically either way.

Q: Does the UI tell the user when something is simulated?

A: That's the PRD's explicit design principle (FR-22, "Demo integrity") — simulated integrations should carry a persistent "Simulated" indicator. Worth checking the live UI for this label on the auto-apply flow specifically if asked to demo it, since that's the one with no real backend at all.

8. Security

Q: How are API credentials protected?

A: All third-party credentials (OpenAI, eGov Face Liveness, eGovPay, eMessage, Supabase service role) live only in server environment variables and are read only inside `"server-only"`-tagged modules called from Route Handlers — never sent to or bundled into client JavaScript.

`NEXT_PUBLIC_SUPABASE_URL` is the one intentionally public value (a project URL, not a secret); the paired service-role key is not public.

Q: What stops a user from tampering with fee amounts or payment requests client-side?

A: The payment route never trusts client-submitted amounts or step selections at face value — it re-derives the journey and fee bill server-side from the cached/generated source of truth and only proceeds with steps that independently validate as payable (see §5).

Q: Is there rate limiting / abuse protection on the AI generation endpoint?

A: Not implemented today — the main practical guard against runaway OpenAI cost is the 24-hour Supabase cache per (event, language), which means repeated requests for the same life event don't re-trigger generation. There's no per-IP/per-session throttle; that would be a reasonable "what's next" answer if pressed.

Q: What's the biggest known security gap right now?

A: The eGovPay webhook has no signature verification yet (documented TODO — eGovPay's spec doesn't define one), so it shouldn't be trusted as an authoritative event source; the app correctly avoids relying on it by confirming payment via a direct polled GET instead. Being able to name this unprompted is a stronger answer than claiming everything is secured.

9. Testing & Quality

Q: How is the app tested?

A: Vitest + Testing Library, with 29 test files across `lib/` (pure logic: fee parsing, journey requirements caching, id-wallet, i18n, application queue, ask-step) and `components/` (component-level tests for cards, forms, banners, list screens). Run via `npm test` (or `test:watch`).

Q: What kinds of things are unit-tested vs. left to manual/demo verification?

A: Deterministic logic — fee parsing/billing rules, caching/staleness behavior, data transforms, component rendering states — is unit-tested. The live third-party integrations (real eGovPay sandbox calls, real Face Liveness capture, real OpenAI web-search output) are inherently harder to unit test and are exercised manually against sandbox credentials instead.

10. Hard / Curveball Questions

Q: What happens if two people (or two tabs) hit the application-queue "accept" flip at the same time?

A: `getQueueState` guards its update with `.eq("status", "pending")`, so only one concurrent request actually flips the row; a losing concurrent request that finds no row from its own update falls back to treating the step as already-accepted rather than erroring — the code comment calls this out explicitly.

Q: Why a 24-hour cache window for journey content, and what if a fee changes the same day?

A: It's a pragmatic trade-off between cost/latency (avoiding an LLM+web-search round trip on every visit) and freshness; a same-day fee change wouldn't be picked up until the next natural regeneration. Nothing currently forces an early bust — an admin/manual invalidation endpoint would be the natural next step if this mattered for a real deployment.

Q: This only demos one life event well (having a baby) per the PRD — is the automation engine actually generic, or hardcoded to that scenario?

A: The pipeline itself is life-event-agnostic: journey generation, fee billing, payment, and the application queue all operate on whatever steps a given `eventId` produces, not on a baby-specific schema. "Had a baby" is the PRD's chosen worked example/demo scenario for judging purposes, not a special code path.

Q: What would break first at real scale (thousands of concurrent users)?

A: Likely candidates: (1) no rate limiting on the OpenAI generation path if many distinct/rare life-event descriptions bypass the cache; (2) the eGovPay webhook's lack of signature verification becomes a bigger liability under real traffic/adversarial conditions; (3) localStorage-only persistence means no server-side analytics/support visibility into a citizen's in-progress journey if something goes wrong and they need help.

Q: If you had one more sprint, what would you build next?

A: Reasonable, defensible answers: eGovPay webhook signature verification once eGovPay documents one; an admin path to force-invalidate a stale journey cache entry; the PRD's eGovChain tamper-evident receipt anchoring (currently unimplemented); and replacing the fixed-delay mock auto-apply queue with a real agency integration if/when one becomes available.

Prep tip: for anything marked "simulated"/"mocked" above, lead with *why* (no real endpoint exists, or a live dependency is unreliable during dev) rather than treating it as a shortcoming — the PRD itself mandates labeling simulated integrations, so judges expect to see the boundary named clearly.